

SIMATS ENGINEERING

Assessment Tool 3: Reverse Engineering Task

Course Name	Cloud Computing and Big Data Analytics
Course Code	CSA15
Course Outcome	CO4: Analyze big data characteristics and challenges across domains including security and application aspects. (BL4)
Bloom's Taxonomy Level	Articulation (Level 4)
Question Count	5
Total Marks	30

Code of Conduct

I KAVYA K(192572065) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: KAVYA K

Reverse Engineering Task — Answers

For each task, the described big data solution is analyzed to infer its likely architecture, components, and design decisions, based on standard patterns used in industry-scale big data systems.

Question 1 — Large-Scale Clickstream, Mobile & Transaction Analytics Platform

Billions of daily clicks, mobile interactions, and transactions point to a platform built around a schema-on-read data lake feeding a layered (bronze/silver/gold) processing pipeline, rather than a single monolithic database.

Probable storage & ingestion:

The base layer is almost certainly a distributed object store or HDFS-backed data lake (e.g., Amazon S3, Azure Data Lake Storage, or on-prem HDFS) organized as a landing/raw zone. Ingestion is split by data velocity: high-throughput event data (clicks, app interactions) arrives through a distributed pub-sub log such as Apache Kafka or Amazon Kinesis, while periodic batch extracts from operational transaction databases are pulled in via Sqoop, Apache NiFi, or change-data-capture tools like Debezium. This dual-path ingestion explains how the platform copes with both continuous event volume and discrete transactional updates.

Batch processing & indexing:

Apache Spark (with Hive or Presto/Trino for SQL access) is the likely batch engine, since it scales horizontally across commodity clusters and handles both structured and semi-structured formats. Structured transaction data is normalized into columnar formats (Parquet/ORC) and registered in a central metastore (Hive Metastore or AWS Glue Catalog) for schema governance. Semi-structured clickstream logs (JSON/Avro) are ingested schema-on-read into a raw zone, then cleaned and flattened into curated Parquet tables during the silver stage — this is the standard medallion (bronze-silver-gold) pattern used to separate messy raw data from analytics-ready tables. Indexing for fast lookup is layered on top with a search engine such as Elasticsearch or Apache Solr for ad-hoc querying of curated datasets.

Stream processing for real time:

Given the emphasis on real-time reporting, a stream-processing engine like Apache Flink or Spark Structured Streaming is the probable choice, consuming directly from Kafka to compute rolling metrics (sessions, conversion counts, anomaly flags) with low latency, writing results to a fast serving store (Redis, Apache Druid, or Cassandra) for dashboards.

Scalability & partitioning:

Horizontal scale-out is achieved through key- and time-based partitioning (e.g., partitioning event tables by date/hour and bucketing by user ID), replication across nodes for fault tolerance, and cluster resource management via YARN or Kubernetes with autoscaling. Partition pruning and columnar storage minimize I/O for large aggregate queries, which is essential at billions-of-events scale.

Dashboards & BI connectivity:

The serving/gold layer most likely exposes pre-aggregated tables through a distributed SQL engine (Presto/Trino, BigQuery, or a warehouse such as Snowflake/Redshift). BI tools like Tableau, Power BI, or Looker connect via JDBC/ODBC to this layer, while near-real-time operational dashboards likely query the low-latency store (Druid/Elasticsearch) directly, giving business users both historical and live views without touching raw data.

Question 2 — E-Commerce Behavioral & Personalization Platform

Tracking cart adds, abandonment, product views, and payments across regions implies an event-driven architecture where behavioral streams and transactional records converge on a customer profile used for real-time personalization.

Event streaming & profiling:

Apache Kafka (or a managed equivalent like Kinesis/Event Hubs) is the probable backbone for capturing every browsing and cart event as it happens. Customer profiles are likely stored in a low-latency NoSQL store — Cassandra, DynamoDB, or MongoDB — chosen for its ability to serve millisecond profile lookups during personalization at scale, something a relational database would struggle to do under this volume.

Recommendation workflow:

Recommendations likely follow a two-speed design: offline model training on historical clickstream and purchase data using Spark MLlib or a distributed deep-learning framework, and an online inference service that scores candidates in real time using a feature store populated from the streaming layer. This explains how recommendations stay relevant to a user's most recent session, not just historical behavior.

Synchronizing sessions, clicks & transactions:

Sessionization (grouping raw events into user sessions) is probably done with windowed stream processing in Flink or Spark Structured Streaming, while CDC tools (Debezium) replicate order/payment status changes from the transactional database into the same streaming fabric, allowing session, clickstream, and order data to be joined in near real time for personalization triggers.

Caching, query engines & ML stages:

Redis or Memcached almost certainly caches hot data — active carts, session state, and top recommendations — to keep page response times low. Distributed query engines such as Presto/Trino or BigQuery support ad-hoc analytical queries across the merged dataset. ML stages likely span feature engineering, offline training, online scoring, and continuous A/B testing to validate recommendation quality.

Merging structured & semi-structured data:

Structured order/payment records and semi-structured browsing logs are probably unified in a data lake using a common event schema, then modeled into a star schema (fact tables for orders/events, dimension tables for products/customers) inside the warehouse layer for consistent cross-domain analytics.

Scalability, fault tolerance & dashboards:

Kafka topic partitioning with consumer groups, replicated brokers, and idempotent/exactly-once processing semantics are the likely fault-tolerance mechanisms, since losing cart or payment events would directly harm revenue. KPI dashboards are probably powered by a real-time OLAP store (Apache Druid or ClickHouse) feeding tools like Looker or Grafana for conversion-rate and abandonment monitoring.

Question 3 — Healthcare Platform for Patient Histories, Imaging & Sensor Streams

Combining patient histories, diagnostic images, wearable streams, and prescriptions across multiple heterogeneous data types strongly suggests a polyglot-persistence architecture rather than a single database technology.

Hybrid database architecture:

Structured records (patient demographics, prescriptions, billing) most likely sit in a relational database such as PostgreSQL or Oracle for transactional integrity. Semi-structured clinical notes probably live in a

document store (MongoDB), diagnostic images (DICOM files) are stored in object storage or a PACS (Picture Archiving and Communication System) backed by cloud blob storage, and continuous wearable sensor readings are likely held in a time-series database such as InfluxDB or TimescaleDB, optimized for high-frequency writes and range queries.

Data integration across institutions:

Given the need to connect hospitals, labs, and insurers, the platform almost certainly relies on healthcare interoperability standards like HL7 and FHIR, orchestrated through ETL/integration tools such as Apache NiFi, Talend, or Mirth Connect, with API gateways mediating access between external partner systems and the internal data lake.

Streaming & ML for prediction:

Kafka is the likely transport for continuous sensor data, feeding a stream processor (Spark Structured Streaming or Flink) that computes rolling vitals and triggers alerts. Disease-prediction and monitoring models are probably built with TensorFlow or PyTorch, trained on historical Spark-processed data and deployed as scoring microservices consuming the live stream.

Privacy, encryption & compliance:

Given HIPAA/GDPR-level sensitivity, encryption at rest (AES-256) and in transit (TLS) is essentially mandatory, alongside role-based access control, field-level tokenization or de-identification of PHI, comprehensive audit logging of every access event, and data-masking for non-clinical analytics use cases — these controls are what make sharing data across hospitals, labs, and insurers legally viable.

Clinical decision & population health pipeline:

A curated analytics layer (gold-tier tables in the data lake or a clinical data warehouse) probably feeds both point-of-care clinical decision support (real-time risk alerts surfaced to physicians) and batch population-health analytics (cohort studies, outcome trend dashboards) used by public-health teams, closing the loop between individual care and system-wide insight.

Question 4 — Smart City Sensor, CCTV & Transport Platform

Continuous ingestion from traffic sensors, CCTV metadata, weather feeds, and transport systems implies an IoT-scale architecture where filtering happens close to the source and geospatial context is central to every query.

Edge computing & messaging:

Given the sheer volume and physical distribution of sensors and cameras, the platform likely performs first-pass filtering and feature extraction at the edge (edge gateways or on-device inference) to cut bandwidth before transmission — sending only metadata/events rather than raw video, for instance. A lightweight IoT messaging protocol (MQTT) probably feeds into a distributed broker like Kafka for city-wide aggregation and durable buffering.

Centralized storage design:

Centralized storage is probably a data lake (HDFS or cloud object storage) for raw and historical data, paired with a purpose-built time-series/geospatial database for operational queries — this split lets the platform retain full history cheaply while keeping recent, query-heavy data fast to access.

Geospatial & streaming analytics:

Congestion prediction likely uses geospatial stream processing (Apache Sedona/GeoSpark on Spark, or Flink with geo-windowing) to correlate sensor readings by location and time window, combined with historical traffic-pattern models for short-term forecasting.

Time-series & location-aware databases:

Databases such as InfluxDB, TimescaleDB, or a geo-indexed store (PostGIS, Elasticsearch with geo-queries) are the probable choice for handling both time-series sensor metrics and spatial queries like 'which routes are congested right now within this district.'

Combining batch & real-time analytics:

A Lambda-style architecture is likely in play: a speed layer (Flink/Kafka Streams) drives live congestion alerts and dashboards, while a batch layer (Spark on the data lake) periodically recomputes long-term traffic and urban-planning models, with results merged in a serving layer for consistency.

Security & visualization:

Device-level authentication (certificates/TLS for sensor and camera endpoints), network segmentation, encrypted transport, and strict access control on CCTV metadata are probable safeguards for citizen privacy and infrastructure security. Live operational intelligence is likely delivered through geospatial dashboards (Grafana, Kibana, or a custom GIS front end) fed by the same low-latency store used for alerts.

Question 5 — Population-Scale Healthcare Analytics & Risk Scoring System

Integrating patient records, wearable readings, lab reports, and appointment histories for population-level analytics is architecturally similar to Question 3, but the emphasis here shifts toward distributed processing for cohort-scale analysis and governance for continuous compliance.

Hybrid storage architecture:

As with a clinical EHR system, structured data (records, appointments) probably resides in a relational store, unstructured/semi-structured content (lab report scans, clinical notes) in a document or object store, and high-frequency wearable data in a time-series database — a polyglot design that lets each data type be queried in the way it is actually used.

ETL pipelines & interoperability standards:

Given multi-institution integration, ETL orchestration is likely handled by Apache Airflow or NiFi pipelines that normalize incoming data to FHIR/HL7 formats, resolve patient identities via a master patient index, and load curated data into the analytics layer on a scheduled or event-driven basis.

Real-time monitoring & predictive analytics:

Kafka-fed streaming (Spark Structured Streaming or Flink) most likely powers real-time vital-sign monitoring and threshold-based alerting, while predictive health analytics (deterioration risk, readmission risk) run as periodically retrained batch models scored against the streaming layer for near-real-time risk flags.

Role of distributed processing:

Population-level analysis — cohort comparisons, outcome trends across thousands or millions of patients — almost certainly relies on Spark's distributed in-memory processing to execute otherwise-prohibitive joins and aggregations across the full longitudinal dataset within reasonable time windows.

Encryption, compliance & audit:

Encryption at rest and in transit, strict RBAC/ABAC access policies, de-identification for research use, consent-tracking, and immutable audit logs are the probable governance backbone, since population health analytics still touches individually identifiable data and must remain auditable under regulations like HIPAA or GDPR.

ML for diagnosis & treatment recommendation:

Risk-scoring and treatment-recommendation models (gradient boosting, logistic regression, or deep learning depending on data complexity) are likely deployed as independent scoring microservices behind an API, with a feature store bridging batch-trained features and real-time inputs, and a model-monitoring pipeline tracking drift to trigger retraining — the standard pattern for keeping clinical ML models both accurate and auditable over time.